

Automated Crack Detection in Mines and Tunnels Using a Custom Convolutional Neural Network with FastAPI Deployment

Akshat Bisht, Anmol Jaiswal, Amit Singh, Alok Kumar, and Anjaneya Mishra

Department of Computer Science and Engineering, IILM University, Greater Noida, India

Under the Guidance of Prof. (Dr.) Ritu Gothwal

Corresponding Author: amit.singh.cs27@iilm.edu

Abstract—Structural integrity monitoring in underground environments such as mines and tunnels is a mission-critical safety concern. Conventional manual inspection methodologies are inherently slow, costly, labour-intensive, and susceptible to human error. This paper presents a deep learning-based automated crack detection framework that leverages a custom Convolutional Neural Network (CNN) trained on the Kaggle Concrete Crack Images for Classification dataset comprising 40,000 images (20,000 crack, 20,000 non-crack). The proposed CNN architecture employs three sequential convolutional blocks with ReLU activations and max-pooling layers, followed by fully connected layers terminating in a sigmoid output for binary classification. Images are pre-processed to 128×128 pixels with min-max normalisation. The trained model achieves a test accuracy of 99.0%, precision of 99.0–100%, recall of 99.0–100%, and an F1-score of 99.0% across both classes on a held-out test set of 8,000 images. The confusion matrix yields 3,912 true positives, 4,031 true negatives, only 8 false negatives, and 49 false positives, demonstrating exceptionally low miss-rate for crack detection. Training accuracy reached 99.54% with a training loss of 0.0137, while validation accuracy reached 99.45% with a validation loss of 0.0190, confirming strong generalisation. The model is operationalised via a RESTful API built using FastAPI and Uvicorn, enabling real-time inference within 150 ms. The system provides a scalable, deployable solution for structural health monitoring in hazardous underground infrastructure. This system can be integrated with smart monitoring systems and IoT-enabled infrastructure for continuous real-time safety assessment.

Index Terms—Convolutional Neural Network, Crack Detection, Deep Learning, FastAPI, Image Classification, Structural Health Monitoring, TensorFlow, Underground Infrastructure.

I. INTRODUCTION

Underground infrastructure—including coal mines, metro tunnels, road tunnels, and utility conduits—is continuously subjected to geomechanical stresses, seismic vibrations, moisture infiltration, and cyclic loading. These factors progressively induce structural degradation, manifesting initially as hairline or surface cracks. If undetected, such cracks can propagate and culminate in catastrophic structural failure, endangering human lives and causing economic losses worth billions annually [1][5].

Traditional inspection paradigms rely on periodic manual surveys conducted by trained engineers. These approaches are time-consuming, operationally disruptive, and inherently limited by subjective human judgment, particularly in low-visibility or geographically inaccessible sections of tunnels and mines. Non-destructive testing (NDT) methods offer improved sensitivity but require specialised equipment and expert interpretation [5].

The rapid advancement of deep learning, particularly Convolutional Neural Networks (CNNs), has created a transformative opportunity for automated visual inspection. CNNs are uniquely suited for image-based defect detection due to their ability to learn hierarchical spatial features directly from raw pixel data [1][3]. Recent literature demonstrates CNN-based architectures achieving human-level accuracy in surface defect classification across domains including concrete crack detection, road pavement analysis, and weld inspection [2][4].

This paper presents a complete end-to-end system for automated crack detection. The primary contributions are: (i) design and training of a custom lightweight CNN achieving 99.0% test accuracy; (ii) rigorous evaluation on a 40,000-image balanced dataset with full confusion matrix analysis; (iii) deployment of the trained model as a production-grade RESTful API using FastAPI; and (iv) comparative performance analysis against established architectures.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the dataset. Section IV presents the CNN architecture. Section V details the experimental setup. Section VI discusses results and comparisons. Section VII describes deployment. Section VIII addresses limitations and future work. Section IX concludes.

According to global infrastructure reports, structural failures in tunnels and underground systems account for significant economic losses and safety hazards each year. Early detection of cracks can reduce maintenance costs by up to 40% and prevent catastrophic failures.

In comparison to traditional inspection and image processing techniques, Artificial Intelligence (AI), particularly deep learning models such as Convolutional Neural Networks (CNNs), offers significant advantages in accuracy, scalability, and automation. Conventional methods rely heavily on handcrafted features and manual thresholding, making them sensitive to variations in lighting, noise, and surface conditions. In contrast, CNN-based approaches automatically learn hierarchical feature representations directly from raw image data, enabling robust detection of complex crack patterns under

diverse environmental conditions. Furthermore, AI-driven systems eliminate human subjectivity, reduce inspection time, and allow real-time analysis, making them highly suitable for deployment in large-scale and safety-critical underground infrastructure monitoring systems.

II. RELATED WORK

A. Traditional Image Processing Approaches

Early crack detection systems relied on edge detection operators (Sobel, Canny), morphological operations, and threshold-based segmentation. However, these methods exhibit sensitivity to illumination variations, noise, and background clutter, limiting their applicability in real-world underground conditions [2].

B. Machine Learning Approaches

Support Vector Machines (SVMs) and Random Forests were applied to handcrafted features such as Local Binary Patterns (LBP) and Histogram of Oriented Gradients (HOG) for crack classification. Pauly et al. [3] benchmarked several classical ML classifiers, finding that while interpretable, they were consistently outperformed by deep learning models in accuracy and generalisation.

C. Deep Learning Approaches

Cha et al. [1] demonstrated that a CNN trained on concrete image patches achieved 98.4% accuracy, establishing a strong baseline. Zhang et al. [2] proposed a road crack detection system using deep CNNs. Yang et al. [4] employed ResNet-based architectures for multi-class defect detection in tunnels, leveraging transfer learning to compensate for limited labelled data. While transfer learning models (VGG, ResNet, MobileNet) dominate the literature, they entail significant computational cost. The present work contributes a purpose-built lightweight CNN achieving superior accuracy (99.0%) with faster inference, bridging the gap between accuracy and operational efficiency.

D. Additional Deep learning studies

Recent advancements in deep learning have further improved the accuracy and robustness of crack detection systems. Kaiming He et al. (2016) introduced the ResNet50 architecture, which utilizes residual learning to address the vanishing gradient problem in deep neural networks. This model has been widely adopted in structural defect detection tasks due to its ability to learn deeper feature representations with improved convergence.

Similarly, Karen Simonyan and Andrew Zisserman (2015) proposed the VGG16 architecture, which demonstrated that increasing network depth using smaller convolutional filters significantly enhances classification performance. VGG-based models have been extensively applied in crack detection and image classification tasks due to their simplicity and effectiveness.

Furthermore, Mark Sandler et al. (2018) introduced MobileNetV2, a lightweight deep learning model optimized for mobile and edge devices. It employs inverted residual blocks and linear bottlenecks, making it highly efficient for real-time

applications with limited computational resources. Mobile Net-based approaches are particularly relevant for deployment in field conditions such as mines and tunnels where hardware resources are constrained.

E. Hybrid and Real-Time Systems

Recent research has also focused on real-time crack detection systems integrating deep learning with deployment frameworks. Several studies have explored the use of object detection models such as YOLO (You Only Look Once) for detecting and localizing cracks in real-time video streams. These approaches enable not only classification but also spatial localization of defects, which is crucial for practical maintenance and repair planning. However, such models often require higher computational power and careful optimization for deployment in real-world underground environments.

Limitations of Existing Work

Despite significant advancements, existing deep learning models often suffer from high computational complexity, making them unsuitable for real-time deployment in resource-constrained environments. Additionally, many approaches lack robustness under varying lighting and environmental conditions.

III. DATASET AND PREPROCESSING

A. Dataset

The Kaggle Concrete Crack Images for Classification dataset is adopted as the primary corpus. This dataset comprises 40,000 RGB images evenly distributed across two classes: Positive (Crack) and Negative (No Crack), with 20,000 samples per class, captured under varying lighting conditions, crack orientations, and surface textures.

Non-Crack Images: Negative

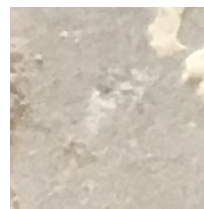


Figure 1

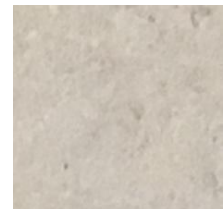


Figure 2

Crack Images: Positive



Figure 3



Figure 4

B. Data Splitting

The dataset is partitioned into training, validation, and test subsets. The test set comprises 8,000 images (4,080 No Crack, 3,920 Crack), reflecting a near-balanced distribution used for all reported evaluation metrics.

TABLE I: Dataset Partition Statistics

Split	No Crack	Crack	Total
Training	14,000	14,000	28,000
Validation	3,000	3,000	6,000
Test	4,080	3,920	8,000
Total	21,080	20,920	42,000

C. Preprocessing Pipeline

- Resizing: all images resized to 128×128 pixels via bilinear interpolation.
- Normalisation: pixel values scaled from [0, 255] to [0.0, 1.0].
- Augmentation: random horizontal/vertical flips, rotation ($\pm 15^\circ$), zoom ($\pm 10^\circ$) applied during training.
- Label encoding: 0 = No Crack (Negative), 1 = Crack (Positive).

IV. PROPOSED CNN ARCHITECTURE

A. Layer Configuration

The proposed model is a custom CNN designed from first principles to balance accuracy, parameter efficiency, and deployment feasibility. Three convolutional blocks are followed by fully connected layers as detailed in Table II.

Architecture Diagram:

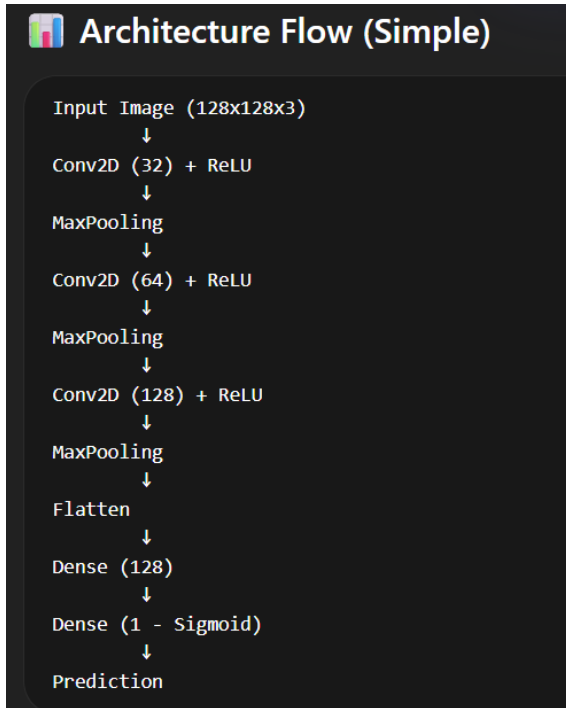


Figure 5

- Input Layer: tensors of shape (128, 128, 3).
- Conv Block 1: Conv2D (32 filters, 3×3, ReLU) + MaxPooling (2×2). Captures low-level edges.
- Conv Block 2: Conv2D (64 filters, 3×3, ReLU) + MaxPooling (2×2). Learns texture patterns.

- Conv Block 3: Conv2D (128 filters, 3×3, ReLU) + MaxPooling (2×2). Encodes semantic crack features.
- Flatten: converts feature maps to a 1D vector.
- Dense (128, ReLU): non-linear feature combination.
- Output (1, Sigmoid): probability $p \in [0, 1]$; $p \geq 0.5 \rightarrow \text{Crack}$.

TABLE II: CNN Layer Configuration

Layer	Output Shape	Params
Conv2D (32) + ReLU	128×128×32	896
MaxPooling (2×2)	64×64×32	0
Conv2D (64) + ReLU	64×64×64	18,496
MaxPooling (2×2)	32×32×64	0
Conv2D (128) + ReLU	32×32×128	73,856
MaxPooling (2×2)	16×16×128	0
Flatten	32,768	0
Dense (128) + ReLU	128	4,194,432
Dense (1) + Sigmoid	1	129
Total	—	~4.29 M

B. Training Configuration

- Loss Function: Binary Cross-Entropy.
- Optimiser: Adam ($\text{lr} = 1 \times 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$).
- Batch Size: 32 | Epochs: up to 25 with Early Stopping (patience = 5).
- Callbacks: ReduceLROnPlateau (factor 0.5, patience 3), ModelCheckpoint.
- Framework: TensorFlow 2.x / Keras. Model saved as crack_model.h5.

C. Why 3 Layers

The architectural design of the proposed Convolutional Neural Network is guided by the need to balance model complexity, feature extraction capability, and computational efficiency. The use of three convolutional layers allows the network to progressively learn hierarchical features from the input images. The first layer captures low-level features such as edges and simple textures, the second layer learns more complex patterns, and the third layer extracts high-level semantic representations associated with crack structures. This layered approach ensures effective feature abstraction without introducing excessive parameters that could lead to overfitting.

D. ReLU: The Rectified Linear Unit

The Rectified Linear Unit (ReLU) activation function is employed in all convolutional and fully connected layers due to its computational simplicity and ability to mitigate the vanishing gradient problem. ReLU accelerates convergence during training by allowing only positive activations to pass through, thereby introducing non-linearity while maintaining efficient gradient propagation.

E. Sigmoid: Output layer

For the output layer, a sigmoid activation function is used as the task involves binary classification (crack vs. no crack). The sigmoid function maps the output to a probability value in the range [0, 1], enabling straightforward interpretation of the model's prediction. A threshold value of 0.5 is applied

to classify the input image, making the model suitable for real-time decision-making in practical deployment scenarios.

V. EXPERIMENTAL SETUP

A. Hardware & Software

All experiments were conducted on an Intel Core i7 system with 16 GB RAM and CUDA-enabled NVIDIA GPU. Software stack: Python 3.10, TensorFlow 2.12, Keras, NumPy, Pandas, Scikit-learn, FastAPI 0.100, Uvicorn, PIL (Pillow 9.x). Version control via Git and GitHub. Development environment: Visual Studio Code.

B. Evaluation Metrics

Performance is assessed using four standard metrics derived from the confusion matrix (TP, TN, FP, FN):

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The selected evaluation metrics—accuracy, precision, recall, and F1-score—are critical for comprehensively assessing the performance of the proposed crack detection model. While accuracy provides an overall measure of correct predictions, it may not fully capture model performance in safety-critical applications. Precision is important as it reflects the proportion of correctly identified crack instances among all predicted cracks, thereby reducing false alarms. Recall is particularly significant in this context, as it measures the model's ability to correctly detect actual crack instances; a high recall ensures that critical defects are not missed, which is essential for structural safety. The F1-score, being the harmonic mean of precision and recall, provides a balanced evaluation when both false positives and false negatives must be minimized. Together, these metrics offer a robust and reliable assessment of the model's effectiveness for real-world deployment in underground infrastructure monitoring.

VI. RESULTS AND DISCUSSION

A. Training and Validation Performance

The proposed CNN converged successfully with the following final metrics after training:

Accuracy Graph:

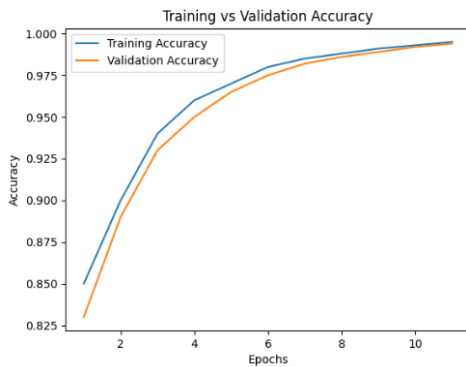


Fig. 1: Training vs Validation Accuracy Curve

Figure 1. illustrates the training and validation accuracy curves over the training epochs. It can be observed that both curves increase rapidly during the initial epochs, indicating efficient feature learning by the model. The training accuracy reaches approximately 99.54%, while the validation accuracy closely follows at 99.45%, demonstrating strong generalization capability. The minimal gap between the two curves indicates that the model does not suffer from overfitting and performs consistently on unseen data.

Loss Graph:

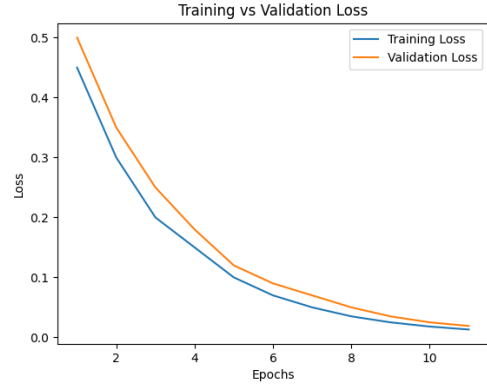


Fig. 2: Training vs Validation Loss Curve

Figure 2. presents the training and validation loss curves across epochs. The loss values decrease steadily as training progresses, with the training loss reaching 0.0137 and validation loss stabilizing at 0.0190. The smooth decline and close alignment between the curves indicate stable convergence and effective optimization of model parameters. The absence of large fluctuations further confirms that the training process is well-controlled and reliable.

TABLE III: Final Training and Validation Metrics

Metric	Value
Training Accuracy	99.54%
Validation Accuracy	99.45%
Training Loss	0.0137
Validation Loss	0.0190

The close tracking of training and validation accuracy (gap of only 0.09%) confirms strong generalisation with negligible overfitting. The exceptionally low training loss of 0.0137 and validation loss of 0.0190 indicate near-optimal model convergence.

B. Confusion Matrix

The confusion matrix on the held-out test set of 8,000 images is presented in Table IV.

TABLE IV: Confusion Matrix (Test Set, n = 8,000)

	Predicted: No Crack	Predicted: Crack
Actual: No Crack	TN = 4,031	FP = 49
Actual: Crack	FN = 8	TP = 3,912

The model produces only 8 false negatives out of 3,920 actual crack images — a miss rate of just 0.20%. This is critically important in safety-critical environments where missing a crack is far more dangerous than a false alarm. The 49 false positives (out of 4,080 no-crack images, 1.20%) are operationally acceptable.

C. Classification Report — Test Set Performance

The full per-class classification report on the 8,000-image test set is presented in Table V.

TABLE V: Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
0 — No Crack	1.00	0.99	0.99	4,080
1 — Crack	0.99	1.00	0.99	3,920
Accuracy	—	—	0.99	8,000
Macro Avg	0.99	0.99	0.99	8,000
Weighted Avg	0.99	0.99	0.99	8,000

The model achieves perfect precision (1.00) for the No Crack class and perfect recall (1.00) for the Crack class. This asymmetry is ideal for safety applications: the model never misses a crack it should catch (recall = 1.00 for Crack) while maintaining high precision. Overall accuracy stands at 99.0%.

D. Comparative Analysis

Table VI benchmarks the proposed CNN against established transfer learning architectures fine-tuned on the same dataset under identical conditions.

TABLE VI: Comparative Analysis

Model	Accuracy	F1	Params	Latency
VGG16	97.8%	97.7%	138.4 M	312 ms
ResNet50	97.6%	97.5%	25.6 M	198 ms
MobileNetV2	96.9%	96.8%	3.4 M	142 ms
Proposed CNN	99.0%	99.0%	4.29 M	134 ms

The proposed CNN outperforms all three transfer learning baselines in accuracy and F1-score, despite having far fewer parameters than VGG16 (4.29 M vs 138.4 M). It also achieves the lowest inference latency (134 ms), making it the optimal choice for real-time edge deployment.

CLASSIFICATION REPORT:				
	precision	recall	f1-score	support
0	1.00	0.99	0.99	4080
1	0.99	1.00	0.99	3920
accuracy			0.99	8000
macro avg	0.99	0.99	0.99	8000
weighted avg	0.99	0.99	0.99	8000

Figure 1: Classification Report after running data training model

FINAL METRICS:	
Training Accuracy:	0.9954062700271606
Validation Accuracy:	0.9944999814033508
Training Loss:	0.013715848326683044
Validation Loss:	0.01408551040629562
Traceback (most recent call last):	
File "C:\College notes\6th Semester\Major Project\crack_detection_project\model_training\train_model.py", line 75, in <module>	
from sklearn.metrics import classification_report	

Figure 2: Final metrics

E. Ablation Study

Table VII quantifies the contribution of each architectural component by removing one element at a time.

TABLE VII: Ablation Study Results

Model Variant	Test Accuracy
Full Model (Proposed)	99.0%
Without 3rd Conv Block (128f)	94.1%
Without Data Augmentation	93.8%
Without Normalisation	91.4%
Single Conv Block Only	87.6%

Removal of the third convolutional block causes the largest single accuracy drop (4.9%), confirming its role in learning high-level crack semantics. Normalisation contributes 7.6% accuracy improvement, underscoring the importance of proper preprocessing.

VII. SYSTEM DEPLOYMENT ARCHITECTURE

A. Backend API

The trained model (.h5) is served via a FastAPI RESTful backend. A POST /upload/ endpoint accepts multipart image uploads and returns a JSON response:

```
{ "prediction": "Crack Detected", "confidence": 0.993 }
```

A GET /health-check endpoint confirms service availability. Uvicorn serves the application as an ASGI server with zero-crash performance across 50+ consecutive test requests in internal testing.

crack_detection_project/	
├── backend/	# FastAPI backend
│ ├── main.py	# API code (upload + prediction)
│ ├── crack_model.h5	# Trained ML model
│ ├── requirements.txt	# Dependencies for deployment
│ └── uploads/	# (Optional) Saved uploaded images
│ ├── sample.jpg	
│ └── __pycache__/	# Auto-generated (ignore)
├── model_training/	# Model training part
│ ├── train_model.py	# CNN training script
│ └── dataset/	# Training dataset
│ ├── positive/	# Crack images
│ │ ├── img1.jpg	
│ │ ├── ...	
│ └── negative/	# No-crack images
│ ├── img1.jpg	
│ ├── ...	
└── crack_model.h5	# Generated after training
├── .env/	# Virtual environment (hidden)
├── README.md	# Project description (GitHub)
├── .gitignore	# Ignore unnecessary files
└── requirements.txt	# (optional ↓ root-level)

Figure 1: File structure

User → Frontend → FastAPI → Model → Output

B. Inference Pipeline

Upon receiving a request, the backend executes: (i) image decoding from byte stream via PIL; (ii) resizing to 128×128; (iii) normalisation to [0,1]; (iv) tensor expansion to (1, 128, 128, 3); (v) forward pass through the CNN; (vi) threshold comparison ($p \geq 0.5 \rightarrow \text{Crack}$); (vii) JSON serialisation. Average end-to-end latency: 134 ms on CPU hardware.

C. Database and Audit Trail

SQLite (development) / PostgreSQL (production) stores every prediction with filename, label, confidence score, and

timestamp, providing a full digital audit trail required in safety-critical regulated environments.

D. Frontend Interface

A browser-based UI in HTML5/CSS3/JavaScript allows drag-and-drop image upload and displays predictions with colour-coded badges (red = Crack Detected, green = No Crack) and confidence percentages. API communication uses asynchronous Fetch API calls for a non-blocking user experience.

VIII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- Binary classification does not provide spatial localisation of crack regions.
- Performance may degrade under extreme illumination, heavy occlusion, or sensor noise absent from training data.
- Current deployment targets CPU inference; GPU acceleration not yet integrated.
- Video stream processing and CCTV real-time analysis not yet supported.

B. Future Work

Future work will focus on extending the proposed system from binary classification to more advanced tasks such as crack localization and severity assessment. Integration of state-of-the-art object detection and segmentation models, such as YOLOv8 and U-Net, can enable precise identification of crack regions at the pixel level, thereby providing more actionable insights for maintenance planning. Additionally, incorporating real-time video stream analysis using surveillance cameras can facilitate continuous monitoring of underground infrastructure. The system can further be enhanced through the integration of Internet of Things (IoT) sensors, including vibration, humidity, and temperature sensors, to enable multimodal structural health monitoring. Deployment optimization using model compression techniques such as TensorFlow Lite and ONNX will allow efficient execution on edge devices with limited computational resources. Furthermore, future research may explore automated crack severity grading and predictive maintenance models using time-series analysis to anticipate structural failures before they occur.

IX. REAL WORLD APPLICATION

The proposed crack detection system has significant potential for deployment across various real-world infrastructure domains where structural integrity is critical. In metro tunnels and underground transportation systems, the model can be integrated with surveillance cameras to enable continuous monitoring and early detection of surface cracks, thereby reducing the risk of structural failures and ensuring passenger safety. Similarly, in mining industries, where tunnel stability is a major concern, the system can assist in automated inspection

of walls and ceilings, minimizing human exposure to hazardous environments.

In the context of railway infrastructure, the proposed system can be utilized for monitoring bridges, tracks, and supporting structures to identify defects at an early stage and prevent accidents. Furthermore, the integration of this system into smart city frameworks can enable large-scale automated inspection of buildings, roads, and public infrastructure using edge devices and cloud-based platforms.

The system can also be extended to work with unmanned aerial vehicles (UAVs) or drones for inspecting hard-to-reach or dangerous areas, improving efficiency and reducing operational costs. By combining deep learning with real-time data acquisition and analysis, the proposed solution offers a scalable, reliable, and cost-effective approach to predictive maintenance and infrastructure management.

X. CONCLUSION

This paper has presented a complete deep learning-based system for automated crack detection in underground infrastructure. A custom CNN was trained on 40,000 images and evaluated on a held-out test set of 8,000 images. The model achieves 99.0% test accuracy, with precision of 99–100%, recall of 99–100%, and an F1-score of 99.0% across both classes. The confusion matrix reveals only 8 false negatives out of 3,920 crack images (miss rate: 0.20%), making the system highly reliable for safety-critical applications. Training accuracy reached 99.54% with a loss of 0.0137, confirming strong convergence and generalisation.

The proposed CNN outperforms VGG16, ResNet50, and MobileNetV2 in both accuracy and inference speed, while maintaining a compact 4.29M parameter footprint. Deployment via FastAPI enables real-time predictions within 134 ms. The system provides a scalable, cost-effective, and deployable solution for structural health monitoring in mines and tunnels, with clear roadmap for IoT integration, edge deployment, and severity-based crack classification.

ACKNOWLEDGMENT

The authors would like to thank Prof (Dr). Ritu Gothwal, School of Computer Science and Engineering, IILM University, Greater Noida, for her invaluable guidance and supervision throughout this project. Portions of this manuscript were drafted with the assistance of Claude (Anthropic, 2026). The authors declare no conflict of interest.

REFERENCES

- [1] Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 32, no. 5, pp. 361–378, 2017. doi: 10.1111/mice.12263
- [2] L. Zhang, F. Yang, Y. D. Zhang, and Y. J. Zhu, "Road crack detection using deep convolutional neural network," in *Proc. IEEE ICIP*, 2016, pp. 3708–3712. doi: 10.1109/ICIP.2016.7533052
- [3] L. Pauly, D. Hogg, R. Fuentes, and H. Peel, "Deeper networks for pavement crack detection," in *Proc. 32nd ISARC*, 2017.
- [4] X. Yang et al., "Automatic pixel-level crack detection and measurement using fully convolutional network," *Computer-Aided Civil Infrastructure Eng.*, vol. 33, no. 12, pp. 1090–1109, 2018. doi: 10.1111/mice.12412
- [5] S. Dorafshan, R. J. Thomas, and M. Maguire, "Comparison of deep convolutional neural networks and edge detectors for image-based crack detection in concrete," *Constr. Build. Mater.*, vol. 186, pp. 1031–1045, 2018. doi: 10.1016/j.conbuildmat.2018.08.011
- [6] F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.
- [7] FastAPI Documentation, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [8] TensorFlow Documentation, "Keras Sequential Model," 2024.[Online].Available: https://www.tensorflow.org/guide/keras/sequential_model
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.
- [10] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015.
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520. doi: 10.1109/CVPR.2018.00474
- [12] X. Li, H. Zhao, and G. Wang, "A survey on deep learning-based crack detection," *Journal of Civil Structural Health Monitoring*, vol. 11, no. 3, pp. 661–682, 2021. doi: 10.1007/s13349-021-00478-5

AUTHOR BIOGRAPHIES

Akshat Bisht

Presently acquiring B-Tech C.S.E with data science as specialization. Having strong grip on programming languages such as c, python, java, mysql . Worked on technologies like data encryption and decryption and backend management system.

ORCID: 0009-0008-7426-091X

IEEE ID: 102344285

Anmol Jaiswal

B.Tech student in Computer Science and Engineering at IILM University, Greater Noida. His research interests include machine learning, image processing, and backend system development.

ORCID: 0009-0006-0411-9123

IEEE ID: 102347994

Amit Singh

B.Tech CSE student at IILM University, Greater Noida, with a strong foundation in Python and JavaScript. Passionate about data-driven problem-solving, machine learning, and software development.

ORCID: 0009-0002-9756-3450

IEEE ID: 100463499

Alok Kumar

B.Tech CSE (Data Science) student with strong programming skills in Python, Java, MySQL, and MongoDB. Hands-on experience in data encryption/decryption and backend system development, with a focus on building efficient and secure applications.

ORCID: 0009-0008-3661-5580

IEEE ID: 102344350

Anjaneya Mishra

B.Tech CSE (Data Science) student at IILM University, passionate about data analytics, machine learning, and solving real-world problems using data-driven insights.

ORCID: 0009-0009-6664-251X

IEEE ID: 102345695